

WARNING

Silent truncation is one of the most common production RAG bugs. A 3,000-token transcript section embedded with a 512-token model is represented only by its **first ~17%**, and everything said later in that section is unretrievable. Always measure token length with the embedder's *own* tokenizer.

Constraint 2: vector dilution (the precision argument). A mean-pooled embedding is approximately an average of the semantic content of its tokens. For a chunk covering m roughly independent topics with directions $t_1 \dots t_m$:

$$e \approx (1/m) \sum_i t_i \Rightarrow \cos(q, e) \approx (1/\sqrt{m}) \cdot \cos(q, t_j) \text{ for a query } q \text{ aligned with one topic } t_j$$

(assuming near-orthogonal unit topic vectors, so $\|e\| \approx 1/\sqrt{m}$). The relevant signal is attenuated by about $1/\sqrt{m}$. A chunk mixing budget, design, and scheduling scores lower for a budget query than a pure budget chunk does. That is why "smaller chunks give more targeted results."

Tokens vs characters vs words. Limits are in *model tokens*. English averages about 0.75 words per token for BPE vocabularies, but ratios vary with jargon, numbers, code, and non-English text. Chunk by the embedder's tokenizer, or keep generous safety margins when chunking by characters.

What a chunk must carry. A chunk is text plus the metadata needed for citation and filtering: `source`, `asset`, `t_start/t_end` or `slide_no`, `speaker(s)`, `modality`, `chunk_index`, `embedding_model`.

⚙️ Production Implementation

```
# token_audit.py: detect silent truncation before embedding
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("BAAI/bge-small-en-v1.5")
tok = model.tokenizer
LIMIT = model.max_seq_length # 512 for this model

def token_len(text: str) -> int:
    return len(tok(text, add_special_tokens=True)["input_ids"])

def audit(texts: list[str]) -> dict:
    lens = [token_len(t) for t in texts]
    over = [i for i, n in enumerate(lens) if n > LIMIT]
    lost = sum(max(n - LIMIT, 0) for n in lens) / max(sum(lens), 1)
    return {"n": len(lens), "max": max(lens), "over_limit": len(over), "token_fraction_lost":
round(lost, 4)}

print(audit([open("transcript_part_a.txt").read()]))
# -> {'n': 1, 'max': 9876, 'over_limit': 1, 'token_fraction_lost': 0.9482} # 95% invisible!
```

⚖️ Trade-offs, Edge Cases & Failure Modes

- **Long-context embedders are not a free pass.** An 8k-token window avoids truncation but *worsens dilution*. Long-context embedding suits document-level similarity, not pinpoint retrieval.